МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ

ОТЧЁТ

по лабораторной работе № 1

«Итерационные методы вычисления элементарных функций: квадратный корень, кубический корень и натуральный логарифм»

Дисциплина: **Теория алгоритмов**

Направление подготовки: 38.03.05 «Бизнес-информатика»

Вариант 13

Выполнила: студентка группы БИ 2-1

Кугушева Арина Вячеславовна

Проверил: _____

2026

1. Цель работы и задание варианта

Цель работы: сформировать представление об итерационных алгоритмах как о вычислительных процедурах, результат которых достигается последовательным уточнением приближений, и освоить приёмы оценки их сходимости и трудоёмкости.

Задание (вариант 13). Вычислить $\sqrt{31}$, $\sqrt[3]{100}$ и $\ln 0{,}2$ с точностью $\varepsilon = 10^{-5}$ без использования `math.sqrt`, `math.log` и `**0.5` внутри вычислительных функций. Корни находятся методом Ньютона, логарифм — разложением в степенной ряд. Критерий остановки — по разности соседних приближений (P). Начальное приближение для методов Ньютона $x_0 = a/2$ (для кубического корня — $x_0 = b/2$). Результаты сравнить с эталонами библиотеки `math`.

№	a	b	c	ε	Критерий	x_0	Доп.
13	31	100	0,2	10^{-5}	P	a/2	Д1

Дополнительное требование Д1: вывести таблицу итераций для каждой функции (номер шага, x_n, $|x_{n+1} - x_n|$) и проанализировать, как меняется число верных знаков от шага к шагу.

2. Краткие теоретические сведения

Итерационный алгоритм строит последовательность приближений $x_{n+1} = \varphi(x_n)$, предел которой равен искомой величине. Он определяется начальным приближением x_0, итерационной формулой φ и критерием остановки. Здесь используется остановка по разности соседних приближений $|x_{n+1} - x_n| < \varepsilon$, а предельное число итераций N_{max} служит страховкой от зацикливания.

Метод Ньютона для уравнения $F(x) = 0$:

$$x_{n+1} = x_n - F(x_n) / F'(x_n).$$

Квадратный корень. $F(x) = x^2 - a$, $F'(x) = 2x$, откуда получается формула Герона:

$$x_{n+1} = \tfrac{1}{2} \cdot (x_n + a / x_n).$$

Кубический корень. $F(x) = x^3 - b$, $F'(x) = 3x^2$:

$$x_{n+1} = \tfrac{1}{3} \cdot (2x_n + b / x_n^2).$$

Для отрицательного b используется тождество $\sqrt[3]{b} = -\sqrt[3]{|b|}$.

Натуральный логарифм. Применяется преобразованный ряд, сходящийся при любом $c > 0$:

$$\ln c = 2 \cdot (y + y^3/3 + y^5/5 + y^7/7 + \ldots),\ y = (c - 1)/(c + 1).$$

Каждый следующий член получается из предыдущего умножением степени на y^2: $t_k = 2y^{2k+1}/(2k+1)$. Суммирование прекращается, когда $|t_k| < \varepsilon$. Для $c = 0{,}2$ имеем $y = -2/3$.

Скорость сходимости. Если $e_{n+1} \approx C \cdot e_n^{\,p}$, то p — порядок сходимости. Метод Ньютона сходится квадратично ($p = 2$): число верных знаков примерно удваивается на каждой итерации. Ряд для логарифма сходится линейно ($p = 1$): число верных знаков растёт на примерно постоянную величину за шаг, определяемую знаменателем $|y|^2$.

3. Блок-схемы алгоритмов

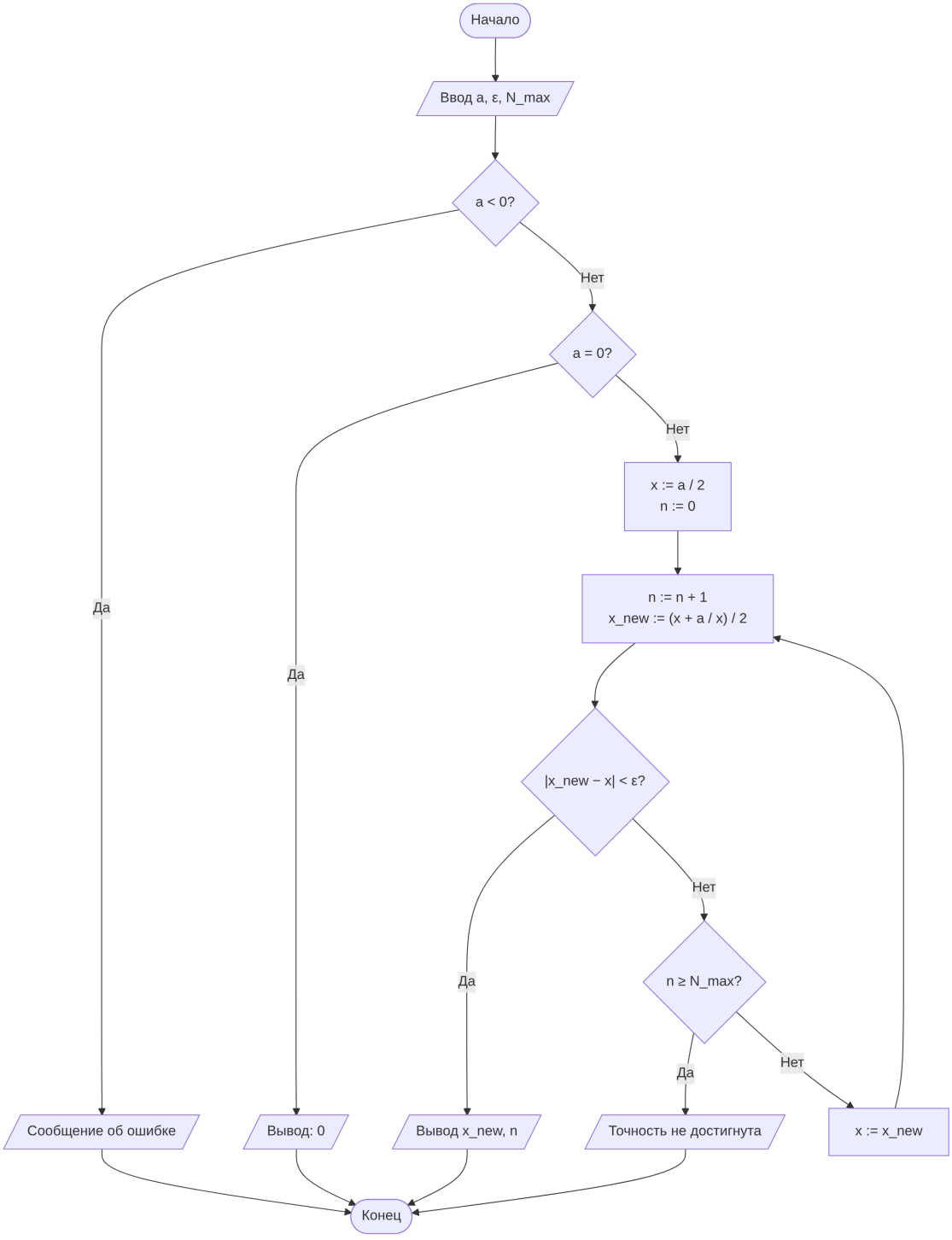


Рис. 1. Блок-схема вычисления √a (метод Ньютона)

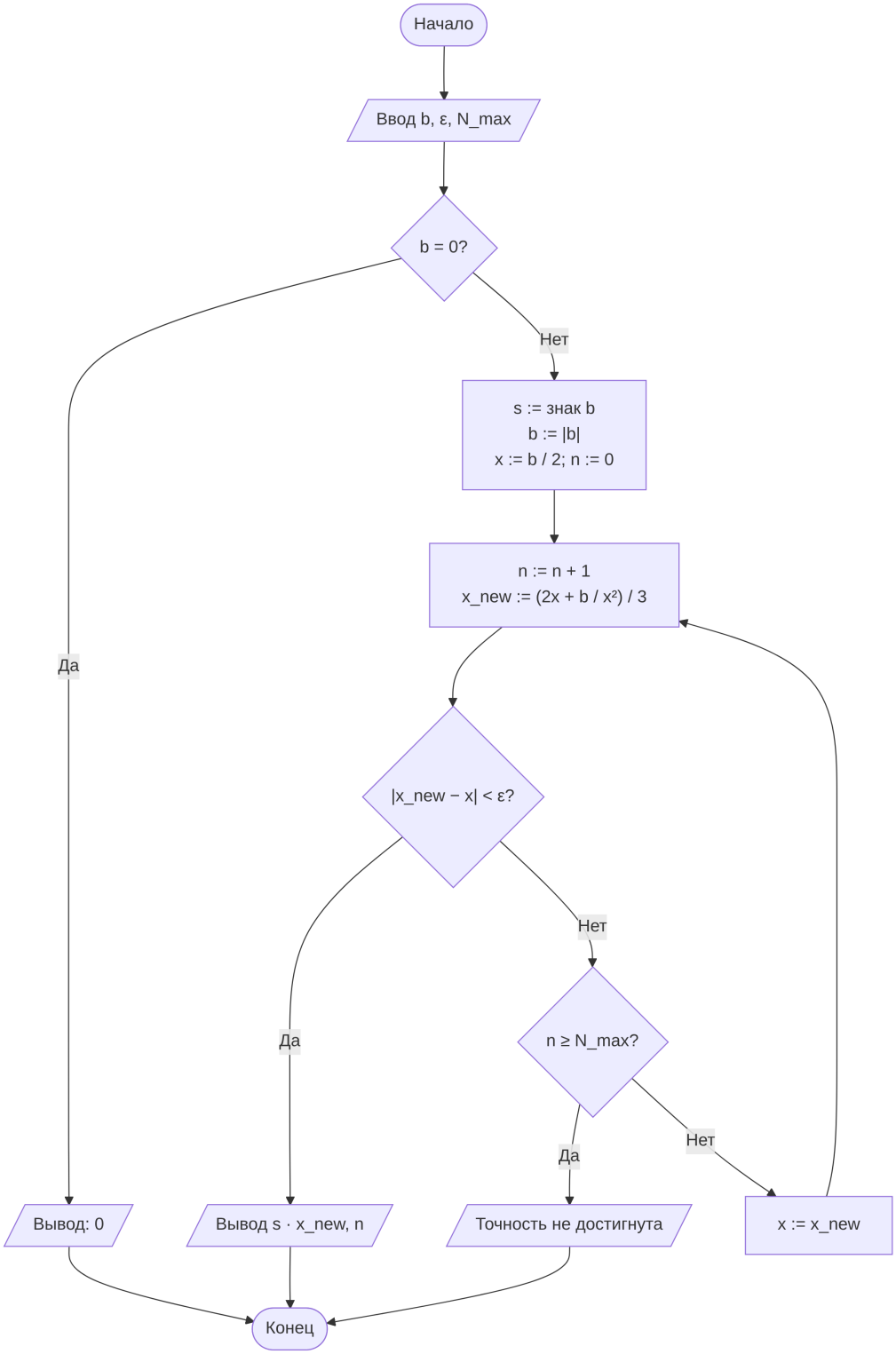


Рис. 2. Блок-схема вычисления $\sqrt[3]{b}$ (метод Ньютона)

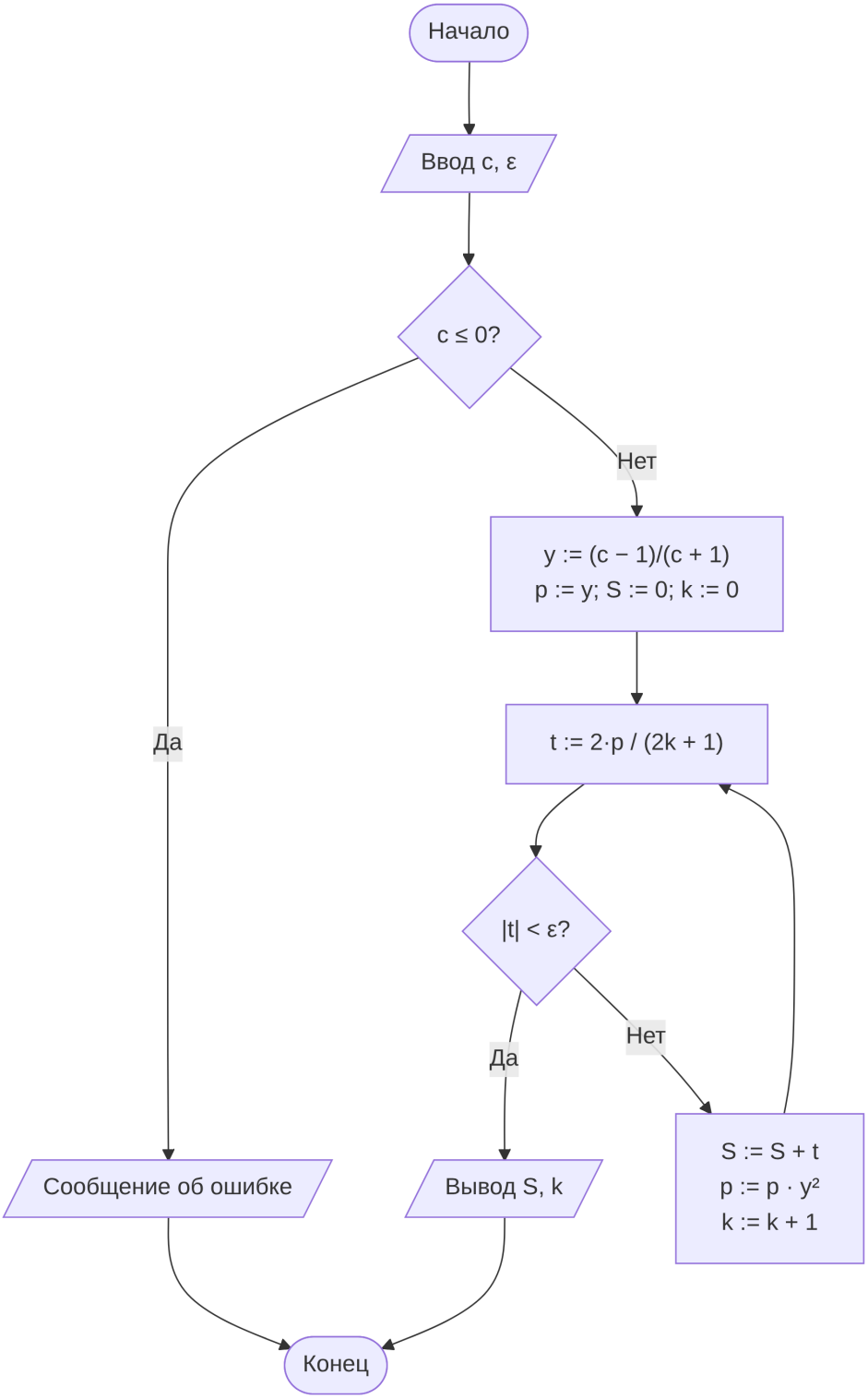


Рис. 3. Блок-схема вычисления ln c (разложение в ряд)

4. Листинг программы

Файл `lab1_variant13.py`. Каждая вычислительная функция возвращает пару «результат, число итераций»; необязательный параметр `trace` собирает данные для таблиц Д1.

```
"""
Лабораторная работа № 1. Вариант 13.
Дисциплина: Теория алгоритмов.
Выполнила: Кугушева Арина Вячеславовна, группа БИ 2-1.

Итерационные методы вычисления элементарных функций:
    sqrt(a)  - метод Ньютона (формула Герона),
    cbrt(b)  - метод Ньютона,
    ln(c)    - разложение в степенной ряд.

Исходные данные варианта 13:
    a = 31, b = 100, c = 0.2, eps = 1e-5,
    критерий остановки - по разности соседних приближений,
    начальное приближение x0 = a / 2,
    дополнительное требование Д1 - таблица итераций для каждой функции
    и анализ изменения числа верных знаков.
"""
import math

A, B, C = 31, 100, 0.2
EPS = 1e-5
N_MAX = 100


# ---------------------------------------------------------------------------
# Вычислительные функции (math.sqrt, math.log, **0.5 внутри НЕ используются)
# Каждая функция возвращает пару (результат, число итераций).
# Если передан список trace, в него добавляются строки таблицы итераций
# (номер шага, приближение, |разность соседних приближений|).
# ---------------------------------------------------------------------------
def sqrt_newton(a, eps=EPS, n_max=N_MAX, trace=None):
    """Квадратный корень методом Ньютона (формула Герона), x0 = a / 2."""
    if not isinstance(a, (int, float)) or isinstance(a, bool):
        raise TypeError("Аргумент должен быть числом")
    if a < 0:
        raise ValueError("Подкоренное выражение должно быть неотрицательным")
    if a == 0:
        return 0.0, 0
    x = a / 2                                   # начальное приближение
    if trace is not None:
        trace.append((0, x, None))
    for n in range(1, n_max + 1):
        x_new = 0.5 * (x + a / x)               # итерационная формула
        diff = abs(x_new - x)
        if trace is not None:
            trace.append((n, x_new, diff))
        if diff < eps:                          # критерий по разности
            return x_new, n
        x = x_new
    raise RuntimeError("Точность не достигнута за N_MAX итераций")


def cbrt_newton(b, eps=EPS, n_max=N_MAX, trace=None):
    """Кубический корень методом Ньютона (с учётом знака), x0 = |b| / 2."""
    if not isinstance(b, (int, float)) or isinstance(b, bool):
        raise TypeError("Аргумент должен быть числом")
    if b == 0:
        return 0.0, 0
    sign = -1 if b < 0 else 1
    b = abs(b)
    x = b / 2                                   # начальное приближение
    if trace is not None:
        trace.append((0, sign * x, None))
    for n in range(1, n_max + 1):
        x_new = (2 * x + b / (x * x)) / 3       # итерационная формула
        diff = abs(x_new - x)
        if trace is not None:
            trace.append((n, sign * x_new, diff))
        if diff < eps:
            return sign * x_new, n
        x = x_new
    raise RuntimeError("Точность не достигнута за N_MAX итераций")


def ln_series(c, eps=EPS, n_max=10_000, trace=None):
```

```
    """Натуральный логарифм: ln c = 2(y + y^3/3 + y^5/5 + ...), y=(c-1)/(c+1)."""
    if not isinstance(c, (int, float)) or isinstance(c, bool):
        raise TypeError("Аргумент должен быть числом")
    if c <= 0:
        raise ValueError("Аргумент логарифма должен быть положительным")
    y = (c - 1) / (c + 1)
    y2 = y * y
    power = y                                   # текущая степень y^(2k+1)
    total = 0.0
    for k in range(n_max):
        term = 2 * power / (2 * k + 1)
        if trace is not None:
            trace.append((k, total, abs(term)))
        if abs(term) < eps:                     # критерий по модулю члена ряда
            return total, k
        total += term
        power *= y2
    raise RuntimeError("Точность не достигнута за n_max итераций")


# ---------------------------------------------------------------------------
# Вспомогательные функции вывода
# ---------------------------------------------------------------------------
def correct_digits(value, ref):
    """Число верных десятичных знаков после запятой: -lg|value - ref|."""
    err = abs(value - ref)
    if err == 0:
        return float("inf")
    return max(0.0, -math.log10(err))


def print_trace(title, trace, ref):
    print(f"\n{title}")
    print(f"{'Шаг':>4}{'x_n':>18}{'|x_n - x_(n-1)|':>20}"
          f"{'Погрешность':>16}{'Верных знаков':>15}")
    for n, x, diff in trace:
        d = "—" if diff is None else f"{diff:.3e}"
        err = abs(x - ref)
        print(f"{n:>4}{x:>18.10f}{d:>20}{err:>16.3e}"
              f"{correct_digits(x, ref):>15.2f}")


def main():
    tasks = [
        ("√a", "sqrt", sqrt_newton, A, math.sqrt(A)),
        ("∛b", "cbrt", cbrt_newton, B, B ** (1 / 3)),
        ("ln c", "ln", ln_series, C, math.log(C)),
    ]

    print("Лабораторная работа № 1, вариант 13")
    print(f"a = {A}, b = {B}, c = {C}, eps = {EPS}, x0 = a/2, критерий: по разности")

    # --- Таблица результатов (аналог табл. 3 из задания)
    print(f"\n{'Функция':<10}{'Аргумент':>10}{'Результат':>16}"
          f"{'Эталон':>16}{'Погрешность':>14}{'Итераций':>10}")
    traces = {}
    for label, name, f, arg, ref in tasks:
        tr = []
        val, n = f(arg, trace=tr)
        traces[name] = (tr, ref)
        print(f"{name:<10}{arg:>10}{val:>16.10f}{ref:>16.10f}"
              f"{abs(val - ref):>14.2e}{n:>10}")

    # --- Д1: таблицы итераций
    print("\n=== Д1. Таблицы итераций ===")
    titles = {"sqrt": f"Квадратный корень √{A} (x0 = a/2)",
              "cbrt": f"Кубический корень ∛{B} (x0 = b/2)",
              "ln": f"Натуральный логарифм ln {C} (частичные суммы ряда)"}
    for name in ("sqrt", "cbrt", "ln"):
        tr, ref = traces[name]
        print_trace(titles[name], tr, ref)

    # --- Д6-подобная проверка входных данных (не обязательна для варианта)
    print("\n=== Проверка обработки ошибочных данных ===")
    for f, arg in [(sqrt_newton, -4), (ln_series, 0), (cbrt_newton, "abc")]:
        try:
            f(arg)
        except (ValueError, TypeError) as e:
            print(f"{f.__name__}({arg!r}): {e}")


if __name__ == "__main__":
    main()
```

5. Результаты расчётов

Таблица 1. Результаты вычислений (вариант 13, $\varepsilon = 10^{-5}$)

Функция	Аргумент	Результат	Эталон (`math`)	Абс. погрешность	Число итераций
$\sqrt{a}$	31	5,5677643628	5,5677643628	$8{,}88 \cdot 10^{-16}$	6
$\sqrt[3]{b}$	100	4,6415888336	4,6415888336	$1{,}28 \cdot 10^{-13}$	10
$\ln c$	0,2	−1,6094247670	−1,6094379124	$1{,}31 \cdot 10^{-5}$	11

Эталон для $\sqrt[3]{100}$ вычислен как $100^{1/3}$ (`b ** (1/3)`), для $\sqrt{31}$ и $\ln 0{,}2$ — функциями `math.sqrt` и `math.log`; эталонные функции используются только для сравнения.

5.1. Таблицы итераций (требование Д1)

Таблица 2. Итерации для $\sqrt{31}$ ($x_0 = 15{,}5$)

Шаг	x_n	$\lvert x_n - x_{n-1} \rvert$	Погрешность	Верных знаков
0	15,5000000000	—	$9{,}93 \cdot 10^{0}$	0,00
1	8,7500000000	$6{,}75 \cdot 10^{0}$	$3{,}18 \cdot 10^{0}$	0,00
2	6,1464285714	$2{,}60 \cdot 10^{0}$	$5{,}79 \cdot 10^{-1}$	0,24
3	5,5950039429	$5{,}51 \cdot 10^{-1}$	$2{,}72 \cdot 10^{-2}$	1,56
4	5,5678306715	$2{,}72 \cdot 10^{-2}$	$6{,}63 \cdot 10^{-5}$	4,18
5	5,5677643632	$6{,}63 \cdot 10^{-5}$	$3{,}95 \cdot 10^{-10}$	9,40
6	5,5677643628	$3{,}95 \cdot 10^{-10}$	$8{,}88 \cdot 10^{-16}$	15,05

Таблица 3. Итерации для $\sqrt[3]{100}$ ($x_0 = 50$)

Шаг	x_n	$\lvert x_n - x_{n-1} \rvert$	Погрешность	Верных знаков
0	50,0000000000	—	$4{,}54 \cdot 10^{1}$	0,00
1	33,3466666667	$1{,}67 \cdot 10^{1}$	$2{,}87 \cdot 10^{1}$	0,00
2	22,2610871255	$1{,}11 \cdot 10^{1}$	$1{,}76 \cdot 10^{1}$	0,00
3	14,9079892640	$7{,}35 \cdot 10^{0}$	$1{,}03 \cdot 10^{1}$	0,00
4	10,0886420142	$4{,}82 \cdot 10^{0}$	$5{,}45 \cdot 10^{0}$	0,00
5	7,0532628641	$3{,}04 \cdot 10^{0}$	$2{,}41 \cdot 10^{0}$	0,00
6	5,3722119659	$1{,}68 \cdot 10^{0}$	$7{,}31 \cdot 10^{-1}$	0,14
7	4,7364493295	$6{,}36 \cdot 10^{-1}$	$9{,}49 \cdot 10^{-2}$	1,02
8	4,6434759942	$9{,}30 \cdot 10^{-2}$	$1{,}89 \cdot 10^{-3}$	2,72
9	4,6415896005	$1{,}89 \cdot 10^{-3}$	$7{,}67 \cdot 10^{-7}$	6,12
10	4,6415888336	$7{,}67 \cdot 10^{-7}$	$1{,}28 \cdot 10^{-13}$	12,89

Таблица 4. Итерации для ln 0,2 (x_k — частичная сумма ряда из k членов, y = −2/3)

Шаг	S_k	$\lvert t_k\rvert$ (следующий член)	Погрешность	Верных знаков
0	0,0000000000	$1{,}33\cdot10^{0}$	$1{,}61\cdot10^{0}$	0,00
1	−1,3333333333	$1{,}98\cdot10^{-1}$	$2{,}76\cdot10^{-1}$	0,56
2	−1,5308641975	$5{,}27\cdot10^{-2}$	$7{,}86\cdot10^{-2}$	1,10
3	−1,5835390947	$1{,}67\cdot10^{-2}$	$2{,}59\cdot10^{-2}$	1,59
4	−1,6002612842	$5{,}78\cdot10^{-3}$	$9{,}18\cdot10^{-3}$	2,04
5	−1,6060417942	$2{,}10\cdot10^{-3}$	$3{,}40\cdot10^{-3}$	2,47
6	−1,6081437978	$7{,}90\cdot10^{-4}$	$1{,}29\cdot10^{-3}$	2,89
7	−1,6089342949	$3{,}04\cdot10^{-4}$	$5{,}04\cdot10^{-4}$	3,30
8	−1,6092387827	$1{,}19\cdot10^{-4}$	$1{,}99\cdot10^{-4}$	3,70
9	−1,6093581896	$4{,}75\cdot10^{-5}$	$7{,}97\cdot10^{-5}$	4,10
10	−1,6094056731	$1{,}91\cdot10^{-5}$	$3{,}22\cdot10^{-5}$	4,49
11	−1,6094247670	$7{,}75\cdot10^{-6}$	$1{,}31\cdot10^{-5}$	4,88

«Верных знаков» — число верных десятичных знаков после запятой, $-\lg\lvert x_n - x^*\rvert$, где x^* — эталонное значение. Для √31 на 6-м шаге значение 15,05 ограничено точностью двойной точности ($\approx 10^{-16}$).

6. Анализ результатов и выводы

1. Все три алгоритма достигли заданной точности $\varepsilon = 10^{-5}$: $\sqrt{31} \approx 5{,}5677643628$ за 6 итераций, $\sqrt[3]{100} \approx 4{,}6415888336$ за 10 итераций, $\ln 0{,}2 \approx -1{,}6094247670$ за 11 членов ряда.
2. **Квадратичная сходимость метода Ньютона** (Д1). Для √31 число верных знаков по шагам равно 0; 0; 0,24; 1,56; 4,18; 9,40; 15,05. На последних шагах оно почти удваивается (1,56 → 4,18 → 9,40 → 15,05), а погрешность падает как $e_{n+1} \approx C\cdot e_n^{\,2}$: $2{,}7\cdot10^{-2} \to 6{,}6\cdot10^{-5} \to 3{,}9\cdot10^{-10} \to 8{,}9\cdot10^{-16}$. Фактическая погрешность ($\approx 10^{-16}$) на много порядков меньше ε, так как к моменту срабатывания критерия по разности погрешность уже пренебрежимо мала: разность $\lvert x_n - x_{n-1}\rvert$ оценивает погрешность *предыдущего* приближения.
3. **Влияние начального приближения.** Выбор $x_0 = a/2$ далёк от корня (для √31: $x_0 = 15{,}5$ при $x^* \approx 5{,}57$; для $\sqrt[3]{100}$: $x_0 = 50$ при $x^* \approx 4{,}64$). Пока x_n значительно больше корня, метод Ньютона ведёт себя почти линейно: для $\sqrt[3]{100}$ каждый шаг уменьшает x примерно в 1,5 раза (50 → 33,3 → 22,3 → 14,9 → 10,1 → 7,05), и первые 5 шагов не дают ни одного верного знака. Квадратичный режим включается только вблизи корня: 1,02 → 2,72 → 6,12 → 12,89 верных знаков на шагах 7–10. Поэтому для $\sqrt[3]{100}$ потребовалось 10 итераций против 6 для √31.
4. **Линейная сходимость ряда для логарифма** (Д1). Для ln 0,2 параметр $y = -2/3$, $|y|^2 = 4/9 \approx 0{,}444$, поэтому отношение соседних членов стремится к y^2 (члены убывают примерно в 2,5 раза за шаг), а число верных знаков растёт почти равномерно — на ≈0,4 за шаг (0,56; 1,10; 1,59; 2,04; … 4,88). Удвоения числа знаков, как у метода Ньютона, нет: для получения точности 10^{-5} потребовалось 11 членов ряда, а для $\varepsilon = 10^{-15}$ потребовалось бы 38 членов (проверено расчётом).
5. **Фактическая погрешность может превышать ε.** Для ln 0,2 абсолютная погрешность ($1{,}31\cdot10^{-5}$) оказалась *больше* $\varepsilon = 10^{-5}$. Критерий $\lvert t_k\rvert < \varepsilon$ гарантирует малость только *очередного* члена ($7{,}7\cdot10^{-6}$), но не суммы всего отброшенного «хвоста» ряда, который для линейно сходящегося процесса сравним с ε и может

быть больше. Для строгой гарантии нужно либо ужесточить критерий (например, $|t_k| < \varepsilon\cdot(1 - y^2)$), либо учитывать оценку хвоста геометрической прогрессии $|t_k|/(1 - y^2)$. У методов Ньютона такой проблемы не возникает из-за квадратичной сходимости.

6. Для повышения эффективности вычисления логарифма аргументов, далёких от 1, целесообразна нормализация $c = m\cdot 2^k$, при которой $|y|$ уменьшается и ряд сходится быстрее.

Вывод. Итерационные методы позволили вычислить $\sqrt{31}$, $\sqrt[3]{100}$ и $\ln 0{,}2$ с заданной точностью без встроенных функций. Метод Ньютона (квадратичная сходимость) значительно эффективнее степенного ряда (линейная сходимость), однако его скорость сильно зависит от выбора начального приближения. Результаты согласуются с эталонными значениями библиотеки `math`.

7. Список литературы

1. Кнут Д. Э. Искусство программирования. Т. 1. Основные алгоритмы. — 3-е изд. — М.: Вильямс, 2017.
2. Вирт Н. Алгоритмы и структуры данных. Новая версия для Оберона. — М.: ДМК Пресс, 2016.
3. Вержбицкий В. М. Основы численных методов. — М.: Высшая школа, 2009.
4. Бахвалов Н. С., Жидков Н. П., Кобельков Г. М. Численные методы. — 9-е изд. — М.: Лаборатория знаний, 2020.
5. ГОСТ 19.701-90 (ИСО 5807-85). ЕСПД. Схемы алгоритмов, программ, данных и систем. Условные обозначения и правила выполнения.